COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	August 28, 2026	**DATE SUBMITTED:**	September 2, 2026
NAME:	Silao, Johrel Louie T.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _6.1_ Searching Techniques

1. PROCEDURE OUTPUT

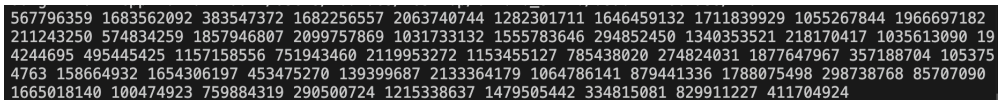
Screenshot	
Observations	Upon implementing and writing the code block, it prints random integers using rand() and stores those random numbers in the array.

Table 6-1. Data Generated and Observations.

Code

```
#include <iostream>
#include <cstdlib> // for generating random integers
#include <time.h> // will be used for our seeding function
#include "searchingAlgo.h"

const int max_size = 50;

int main () {
    int dataset[max_size];
    srand(time(0));
    for (int i = 0; i < max_size; i++){
        dataset[i] = rand();
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

    std::cout << "What number to find?: ";
    int choice;
    std::cin >> choice;

    linearSearchAR(max_size, choice, dataset);

    return 0;
}
```

```cpp
#include "node.h"

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H
/*




*/
template <typename T>
void linearSearchAR(int size, T dataFind, T array[]){
    // while loop implementation
    int i = 0;

    while (i < size) {
        if (dataFind == array[i]) {
            std::cout << "Searching is Successfull!\n";
            return;
        }
        i++;
    }
    std::cout << "Searching is Unsuccessfull!\n";

}

#endif
```

Output

```
(base) lourdes@Dhesss-MacBook-Air dsaOnlineClass % cd "/Users/lourdes/Desktop/CPE010_SILAO/dsaOnlineClass/"
&& g++ main.cpp -o main && "/Users/lourdes/Desktop/CPE010_SILAO/dsaOnlineClass/"main
583863851 1146960614 1153824026 543072572 615217854 1980195520 1592027081 1700392394 1930075329 1035566568 1559833088 1781831087 575621794 61662023 1266502707 253087485 1623739335 2112300916 1389326655 823396754 444623210 1686682557 1289595099 1819863369 1981542209 603508987 610279728 589490424 1223492557 1083485474 1626518605 1578851572 1450428272 1261090407 1630358206 1686516169 640595630 1155230999 567747666 865178841 449006850 200592392 1954490201 1206943695 2119635950 115191567 1141900622 2009884362 208704824 861181417
What number to find? 50
Searching is Unsuccessfull!
(base) lourdes@Dhesss-MacBook-Air dsaOnlineClass % cd "/Users/lourdes/Desktop/CPE010_SILAO/dsaOnlineClass/" && g++ mai
n.cpp -o main && "/Users/lourdes/Desktop/CPE010_SILAO/dsaOnlineClass/"main
584267219 1483915649 1442720132 559400247 156544763 380364166 1869204490 181591467 433523482 1964631350 1998026825 635059636 453576662 1843495031 1874410748 1783823793 1854776831 361578765 1823065992 2120935795 486850012 575456614 1580449057 382071256 495495062 1991407615 1055146810 2079962391 1189099671 715351515 1299456699 60050103 2092250678 1559909168 913024000 1423710185 1034284421 1485624929 105818034 367237722 296392176 1448724639 545417987 1379902113 1338909238 1713909800 1483851389 362702312 1379167598 1878817515
What number to find? 584267219
Searching is Successfull!
```

Observations

The template linearSearchAR checks the elements in the random generated integers that are stored in the array by using a while loop. The first run a user inputs 50, then the program prints “Searching is Unsuccessful!” because 50 is not present in the random generated integers. In the second run, a user input 584267219, then the program prints “Searching is Successful!” because the inputted integer identified the existing element.

Table 6-2a. Linear Search for Arrays

Code

```
//Creating a node for your name
Node<char> *char1 = new_node('J');
Node<char> *char2 = new_node('I');
Node<char> *char3 = new_node('M');
Node<char> *char4 = new_node('L');
Node<char> *char5 = new_node('O');
Node<char> *char6 = new_node('R');
Node<char> *char7 = new_node('D');

//Connect the nodes
char1->next = char2;
char2->next = char3;
char3->next = char4;
char4->next = char5;
char5->next = char6;
char6->next = char7;

linearSearchLL(char1, 'D');
```

```
template<typename T>
void linearSearchLL(Node<T> *nodes, T dataFind){
    //repeat the loop until the correct node is not NULL

    while(nodes != nullptr){

        //check if the item = current node data
        if(nodes->data == dataFind){
            std::cout<<"The data: "<< nodes->data<< " was found Successfully!\n";
            return;
        }
        // move on to the next node
        nodes = nodes->next;
    }
    //display unsuccessful
        std::cout<<"Searching is Unsuccessful!\n";
}
```

Output

```
The data: D was found Successfully!
```

Observations

This code block demonstrates a linked list implementation that stores characters to form a name, that is followed by a linear search. The characters are linked to their respective nodes. The template linearSearchLL starts at head node pointer using a while loop, searching for the character ‘D’ which matches the output and prints the “The data: D was found successfully!”

Table 6-2b. Linear Search for Linked List

Code

```
//Creating a node for your name
Node<char> *char1 = new_node('J');
Node<char> *char2 = new_node('I');
Node<char> *char3 = new_node('M');
Node<char> *char4 = new_node('L');
Node<char> *char5 = new_node('O');
Node<char> *char6 = new_node('R');
Node<char> *char7 = new_node('D');

//Connect the nodes
char1->next = char2;
char2->next = char3;
char3->next = char4;
char4->next = char5;
char5->next = char6;
char6->next = char7;

//linearSearchLL(char1, 'D');

//generate a sorted array: (use for loop)
int newArr[max_size];

for (int i = 0; i < max_size; i++ ){
    newArr[i] = i;
}

binarySearchArr(newArr, max_size, 100);
```

```
template<typename T>
void binarySearchArr(T data[], int arrSize, T dataFind){
    int low = 0, high = arrSize - 1, middle;
    while(low <= high){
        middle = (int)((low + high/2));
        if(dataFind == data[middle]){
            std::cout<<"The data was found\n";
            return;
        }
        else if(dataFind < data[middle]){
            high = middle - 1;
        }
        else{
            low = middle + 1;
        }
    }

    std::cout<<"The data was not found\n";
}
```

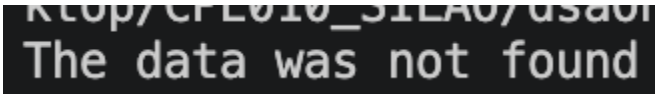
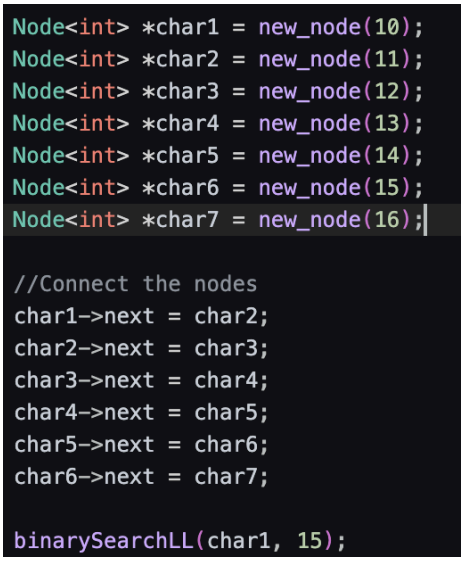
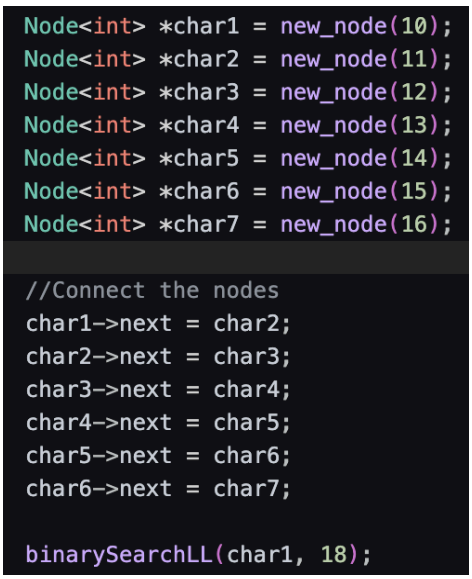
Output	
Observations	This code block shows the binary search implementation on a sorted array of integers. The template binarySearchArr implements the standard binary search on a sorted array using low, high, and middle bounds. When the code runs, the program will call the binarySearchArr and will print “The data was not found” because the target value of 100 is outside the generated range of 0 - 49.

Table 6-3a. Binary Search for Arrays

Code	 

```cpp
//middle function getter:
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){
    Node<T>* slow = start;
    Node<T>* fast = start;

    //continue until fast reaches the end or the last node of the current search range

    while(fast != last && fast->next != last){
        //move slow one step
        slow = slow->next;
        //move fast two steps
        fast = fast->next->next;
    }
    return slow; // return middle
}

template<typename T>
void binarySearchLL(Node<T>* start, T dataFind){
    Node<T>* last = nullptr;

    // Middle element is calculated using the two pointers approach discussed above
    while(start != last){
        Node<T> *middle = getMiddle(start, last);

        //If the middle element is same as the key to be sorted, we return it
        if(middle->data == dataFind){
            std::cout<<"searching is Successful!\n";
            return;
        }
        else if(middle->data > dataFind){
            last = middle;
        }
        else{
            start = middle->next;
        }
    }

    std::cout<<"Searching is unsuccessful\n";
}
```

Output

```
(base) lourdes@Dhesss-MacBook-Air dsaOnlineCla
s/Des
ktop/CPE010_SILAO/dsaOnlineClass/"main
searching is Successful!
(base) lourdes@Dhesss-MacBook-Air dsaOnlineCla
s/Des
ktop/CPE010_SILAO/dsaOnlineClass/"main
Searching is unsuccessful
```

Observations

The program is constructed of sorted single linked list with its integer values with a range of 10 - 16 by manually creating their nodes and is assigned to their pointers. This implements the binary search over the linked list using the pointer (slow and fast) inside the getMiddle

	function to determine the midpoint without accessing it directly. The first run searches for the key '15' and is present in the list which prints the search is successful. The second run searches for the key '18' and this is not present in the list because it is outside the range of 10 - 16 which prints the search is unsuccessful.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

For each provided problem, give a screenshot of your code, the output console, and your answers to the Questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

```cpp
#ifndef SEQUENTIAL_SEARCH_H
#define SEQUENTIAL_SEARCH_H

#include <iostream>

template <typename T>
struct Node {
    T val;
    Node* next;
    Node(T v) : val(v), next(nullptr) {}
};

// ================ Problem 1 ================
template<typename T>
void countLinearArray(T list[], int length, T targetKey) {
    int totalChecks = 0;
    for (int i = 0; i < length; ++i) {
        totalChecks++;
        std::cout << "Check #" << totalChecks << " performed.\n";
        if (list[i] == targetKey) {
            std::cout << "Found target at index " << i << ".\n";
            return;
        }
    }
    std::cout << "Key not found.\n";
}

template<typename T>
void countLinearLL(Node<T>* head, T targetKey) {
    int totalChecks = 0;
    Node<T>* temp = head;
    while (temp != nullptr) {
        totalChecks++;
        std::cout << "Check #" << totalChecks << " performed.\n";
        if (temp->val == targetKey) {
            std::cout << "Found target in list.\n";
            return;
        }
        temp = temp->next;
    }
    std::cout << "Key not found in list.\n";
}

```

Figure 1: Code Snippet of Sequential Search (Problem 1)

Observation of Figure 1:

- This defines two C++ template functions inside the header file. The ‘countLinearArray’ searches through the arrays and the ‘countLinearLL’ is for linked lists. Both functions will increment a counter for every iteration and display the number of comparisons performed during the search.

```cpp
#include <iostream>
#include "SequentialSearch.h"
#include "BinarySearch.h"

int main() {
    int seqData[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int seqSize = sizeof(seqData) / sizeof(seqData[0]);

    std::cout << "========== Problem 1 ==========\n";
    countLinearArray(seqData, seqSize, 18);
```

Figure 2: Code Snippet of mainSupplementary.cpp (Problem 1)

Observation of Figure 2:

- Initializes an integer array that contains the elements and calculates its size and calls it within the main function. This executes the sequential search on the target list to traverse how many checks are needed to find the first instance.

```
========== Problem 1 ==========
Check #1 performed.
Check #2 performed.
Found target at index 1.
```

Figure 3: Output Snippet of mainSupplementary.cpp (Problem 1)

Observation of Figure 3:

- This displays the expected output of the program which confirms that 2 checks or procedures were performed before printing the target value.

```cpp
// ================= Problem 2 =================
template<typename T>
void countOccurrencesArray(T list[], int n, T element) {
    int matchCount = 0;
    for (int index = 0; index < n; ++index) {
        if (list[index] == element) {
            matchCount++;
            std::cout << "Match #" << matchCount << " at index " << index << "\n";
        }
    }
    if (matchCount == 0) {
        std::cout << "Element not found.\n";
    } else {
        std::cout << "Total instances detected: " << matchCount << "\n";
    }
}

#endif
```

Figure 4: Code Snippet of Sequential Search (Problem 2)

Observation of Figure 4:

- This implements the countOccurrencesArray which is a modified sequential search function that iterates through the entire array. It traverses and prints each matching index as well as the total count.

```
std::cout << "\n========== Problem 2 ==========\n";
countOccurrencesArray(seqData, seqSize, 18);
```

Figure 5: Code Snippet of mainSupplementary.cpp (Problem 2)

Observation of Figure 5:

- This calls the countOccurrencesArray which is located in the header file.

```
========== Problem 2 ==========
Match #1 at index 1
Match #2 at index 4
Total instances detected: 2
```

Figure 6: Output Snippet of mainSupplementary.cpp (Problem 2)

Observation of Figure 6:

- The terminal displays the matching index and ends with the 'total instances detected: 2' This shows how a full list iteration accurately identifies all the duplicated instances.

```
#ifndef BINARY_SEARCH_H
#define BINARY_SEARCH_H

#include <iostream>

// ================ Problem 3 ================
template<typename T>
void binarySearchIterative(T arr[], int size, T searchVal) {
    int lowIndex = 0;
    int highIndex = size - 1;

    while (lowIndex <= highIndex) {
        int midIndex = lowIndex + (highIndex - lowIndex) / 2;

        if (arr[midIndex] == searchVal) {
            std::cout << "Key " << searchVal << " located at position " << midIndex << ".\n";
            return;
        }
        if (searchVal < arr[midIndex]) {
            highIndex = midIndex - 1;
        } else {
            lowIndex = midIndex + 1;
        }
    }
    std::cout << "Key " << searchVal << " could not be found.\n";
}
```

Figure 7: Code Snippet of Binary Search (Problem 3)

Observation of Figure 7:

- This figure implements the BinarySearchIterative which is an iterative binary search function that uses low, high and middle index inside a while loop in order to locate the value in a sorted array.

```
int binData[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int binSize = sizeof(binData) / sizeof(binData[0]);

std::cout << "\n========== Problem 3 ==========\n";
binarySearchIterative(binData, binSize, 8);
```

Figure 8: Code Snippet of mainSupplementary.cpp (Problem 3)

Observation of Figure 8:

- This figure initializes the sorted array and calls the binarySearchIterative that is located in the header file.

```
========== Problem 3 ==========
Key 8 located at position 3.
```

Figure 9: Output Snippet of mainSupplementary.cpp (Problem 3)

Observation of Figure 9:

- This terminal outputs the 'Key 8 located at position 3.' which verifies the iterative binary search to perform successfully.

```cpp
// ================= Problem 4 =================
template<typename T>
void binarySearchRecursive(T arr[], int low, int high, T targetVal) {
    if (low > high) {
        std::cout << "Item absent from array.\n";
        return;
    }

    int mid = low + (high - low) / 2;

    if (arr[mid] == targetVal) {
        std::cout << "Item identified at index: " << mid << "\n";
        return;
    }

    if (targetVal < arr[mid]) {
        binarySearchRecursive(arr, low, mid - 1, targetVal);
    } else {
        binarySearchRecursive(arr, mid + 1, high, targetVal);
    }
}

#endif
```

Figure 10: Code Snippet of Binary Search (Problem 4)

Observation of Figure 10:

- This figure implements the binarySearchRecursive that replaces the iterative while loop with recursive function by using the low or high bounds until the target value is found or until conditions are met.

```cpp
    std::cout << "\n========== Problem 4 ==========\n";
    binarySearchRecursive(binData, 0, binSize - 1, 8);

    return 0;
}
```

Figure 11: Code Snippet of mainSupplementary.cpp (Problem 4)

Observation of Figure 11:

- This figure will call the binarySearchRecursive that is located in the header file that verifies that the recursive provides the same search procedure.

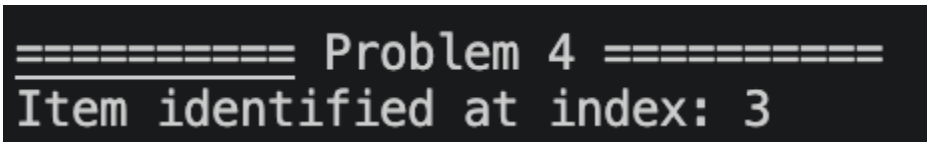


Figure 12: Output Snippet of mainSupplementary.cpp (Problem 4)

Observation of Figure 12:

- The program prints the 'Item identifies at index: 3' which verifies that the recursive binary search is applied correctly and solves the search space and successfully returns the index 3 with the target value of 8.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.